# Q24: How Do We Judge Whether an Agent Is Good? Benchmarks and Metrics

DLS 2026 — Draft module

October 2026

## Learning objectives

Students will distinguish a model score from an agent outcome, choose metrics aligned with a stated deployment goal, identify benchmark leakage and reward-hacking risks, and write an evaluation plan with safety and human-oversight measures.

## 1 An agent is more than a response model

An agent receives an observation, selects actions such as tool calls, sees new observations, and eventually produces an outcome. Evaluation must therefore assess a trajectory, not only whether one final sentence resembles a reference answer. A coding agent might need to find the right repository files, make a minimal change, run tests, and avoid changing unrelated behavior. SWE-bench operationalizes part of this setting using real GitHub issues and repository test suites [2]. WebArena evaluates multi-step web interaction in a controlled environment [4].

A high language-model benchmark score can be relevant, but it is not enough to establish dependable agency. Tool schemas, environment state, retries, context management, permissions, latency, and recovery from failure can determine whether a deployment succeeds. Conversely, an agent can reach a correct end state with an unnecessarily risky or expensive trajectory. The unit of evaluation should match the intended responsibility.

## 2 Start from a task contract

Before choosing a benchmark, state what counts as success, what inputs and tools are permitted, which actions are forbidden, the time and cost budget, and how an evaluator will observe the result. For a support agent, success may combine issue resolution, correct escalation, privacy preservation, and user time. For a research agent, it may combine evidence traceability, calibrated uncertainty, reproducible computation, and avoidance of unsupported conclusions.

The same task can support several metrics. Completion rate asks how often the end state is achieved. Conditional success rate separates attempts that had a usable environment from infrastructure failures. Cost and wall-clock time reveal efficiency. Intervention rate measures how often a human must correct or take over. Constraint-violation rate measures unsafe actions. Reporting only the best metric invites optimization against a narrow proxy.

## 3 Metrics need failure analysis

Consider two agents that each solve 70 of 100 tasks. Agent A spends ten times as much and makes five prohibited tool calls; Agent B fails safely and asks for clarification when required. A single success percentage hides a consequential difference. A useful scorecard reports distributions, not just means: success by task type, tail latency, cost, retry count, and the severity of failures.

A human rubric may be needed for open-ended quality, but it requires blinded examples, rater instructions, inter-rater agreement or adjudication, and clear separation of human preference from factual correctness. Automatic judges can be useful scaling tools, but they should be validated against humans and protected against prompt injection or style-based gaming. Chatbot evaluation research emphasizes that human preference is itself multi-dimensional and noisy, and must be aggregated with explicit statistical models rather than a single universal score [1].

## 4 Benchmark validity and contamination

A benchmark may be too easy, leak into training data, have brittle tests, or reward a shortcut. AgentBench intentionally spans several environments because agent ability does not reduce to one task family [3]. Report uncertainty across seeds and evaluation reruns (variance or confidence intervals), not only point estimates: agent benchmarks are small and stochastic, so a leaderboard gap without a spread is not evidence. Even a diverse suite is only a sample from the target distribution. Keep a private or freshly generated holdout set; vary task wording and environment state; record versions of tools and data; and inspect successes manually.

Avoid repeatedly tuning an agent against the test set. Development tasks are for iteration; an untouched evaluation set is for estimation. When the agent uses the web or retrieval, log the sources and time because the environment can change. A score without the evaluated environment version is difficult to reproduce.

## 5 A minimal evaluation card

For each run, record: task population and sampling method; agent/model and prompt versions; tools, permissions, and environment image; seed and budget; end-state checker; trajectory/constraint checker; costs; failures; and known exclusions. The included toy program demonstrates why success rate alone can select a worse agent when cost and safety violations differ.

## 6 Takeaway

A good agent is good relative to a declared task contract. Evaluate outcomes, trajectories, cost, reliability, and constraints together; preserve a holdout; and inspect failures. A benchmark leaderboard is evidence about its setting, not a universal certificate of agent quality.

## References

[1] Wei-Lin Chiang, Lianmin Zheng, Ying Sheng, Anastasios Nikolas Angelopoulos, Tianle Li, Dacheng Li, Hao Zhang, Banghua Zhu, Michael Jordan, Joseph E. Gonzalez, and Ion Stoica. Chatbot arena: An open platform for evaluating llms by human preference. *arXiv preprint arXiv:2403.04132*, 2024.

[2] Carlos E. Jiménez, John Yang, Alexander Wettig, Shunyu Lierler, Karthik Narasimhan, and Ofir Press. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2024.

[3] Xiao Liu et al. Agentbench: Evaluating llms as agents. *International Conference on Learning Representations*, 2024.

[4] Shuyan Zhou et al. Webarena: A realistic web environment for building autonomous agents. *International Conference on Learning Representations*, 2024.